

Technical System Design Architecture — FlyFF MMO RPG

Document Version: 1.0 **Date:** 2026-06-18 **Project:** Fly For Fun (FlyFF) Clone — MVP
Author: Principal Game Architect

Table of Contents

1. [Executive Summary](#)
 2. [System Architecture Overview](#)
 3. [Technology Stack Deep Dive](#)
 4. [Game Systems Design](#)
 5. [Network Architecture \(Colyseus\)](#)
 6. [Client Architecture \(PlayCanvas + TypeScript\)](#)
 7. [Security & Anti-Cheat](#)
 8. [Deployment Architecture](#)
 9. [Code File Reference](#)
-

1. Executive Summary

1.1 Project Vision

This document outlines the complete technical architecture for a massively multiplayer online role-playing game (MMORPG) inspired by **Fly For Fun (FlyFF)**. The game features a vibrant 3D world where players fly on brooms, select character classes, level up through combat and questing, trade items, form parties, and compete on leaderboards.

1.2 MVP Feature Set

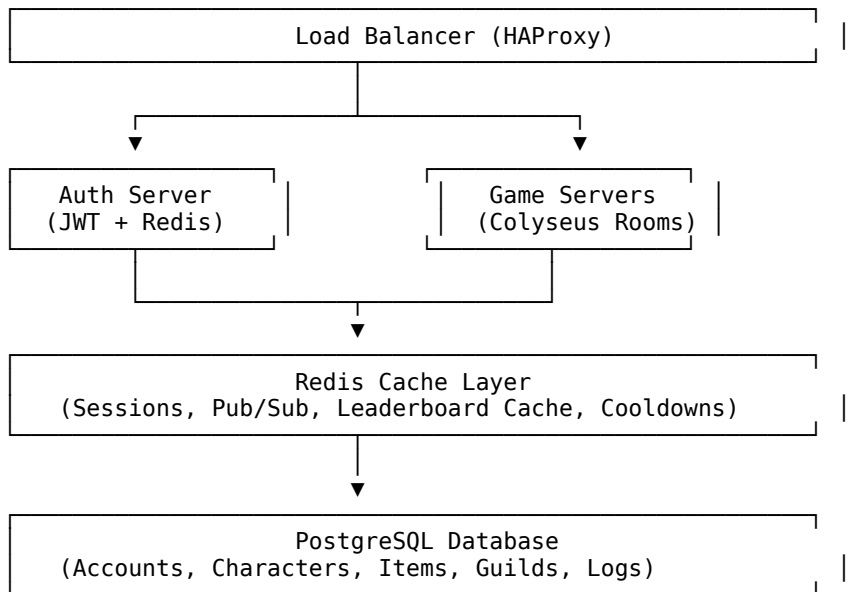
Category	Features
World	Persistent 3D open world, zones, NPCs, day/night cycle
Character	Classes (Assist, Mercenary, Acrobat, Magician), leveling, stats, skills
Combat	Real-time targeted combat, skill system, auto-attack, AoE
Loot & Inventory	Item database, inventory grid, equipment slots, trading
Social	Party system, chat (global/party/whisper), friends list
Lobby & Matchmaking	Character select, channel system, world selection
Economy	Currency (Penya), NPC shops, player trading
Flight	Broom/flying board system, aerial combat
UI/UX	HUD, minimap, skill bar, inventory panel, character sheet
Dashboard	Player stats, leaderboards, achievement tracking
Security	Server-authoritative logic, encryption, anti-cheat

1.3 Target Metrics

Metric	Target
Concurrent Players	500-1000 per server
World Update Rate	20 Hz
Client Frame Rate	60 FPS
Network Latency	< 100 ms
Server Startup	< 30 seconds
Database Write Latency	< 50 ms

2. System Architecture Overview

2.1 High-Level Architecture Diagram



2.2 Component Interaction Flow

1. **Client** connects via WebSocket to Load Balancer
2. **Load Balancer** routes to Auth Server for authentication
3. **Auth Server** validates credentials, issues JWT token
4. **Client** connects to Game Server with JWT
5. **Game Server** creates/joins Colyseus Room
6. **Game Server** loads persistent data from PostgreSQL
7. **Game Server** publishes events via Redis Pub/Sub for cross-server communication
8. **Client** receives state sync at 20 Hz via Colyseus state patches

2.3 Server Architecture (Microservices)

Service	Role	Tech
Auth Service	Login, registration, token management	Node.js + Express + JWT
Game Service	Core gameplay, Colyseus rooms	Node.js + Colyseus
Chat Service	Real-time chat, moderation	Node.js + Colyseus
Lobby Service	Channel management, world list	Node.js + Colyseus
Leaderboard Service	Ranking, stats aggregation	Node.js + Redis Sorted Sets
API Gateway	REST endpoints for profile, items	Node.js + Express

3. Technology Stack Deep Dive

3.1 TypeScript

Role: Primary programming language for both client and server.

Strict Mode Configuration:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "declaration": true,
    "sourceMap": true,
    "outDir": "./dist"
  }
}
```

Usage across stack:

- Shared type definitions between client and server
- Colyseus room schemas
- PlayCanvas script components
- REST API controllers and middleware
- Database ORM models

3.2 Node.js

Role: Server runtime.

Runtime Configuration:

- **Version:** 20 LTS (Active LTS)
- **Process Manager:** PM2 cluster mode
- **Memory:** 4 GB per instance
- **Event Loop:** Offload CPU-intensive tasks to worker threads

Key packages:

```
{
  "dependencies": {
    "colyseus": "^0.15.0",
    "@colyseus/ws-transport": "^0.15.0",
    "@colyseus/auth": "^0.15.0",
    "@colyseus/monitor": "^0.15.0",
    "express": "^4.18.0",
    "redis": "^4.6.0",
    "pg": "^8.11.0",
    "typeorm": "^0.3.0",
    "jsonwebtoken": "^9.0.0",
    "bcrypt": "^5.1.0",
    "socket.io": "^4.6.0",
    "winston": "^3.9.0",
    "node-cron": "^3.0.0"
  }
}
```

3.3 Colyseus

Role: Real-time multiplayer game server framework.

Architecture decisions:

- **Room-based architecture** — each game world/channel is a Room
- **State synchronization** — Schema-based delta compression
- **Simulation** — Server-authoritative game loop running at 20 Hz
- **Authentication** — @colyseus/auth integration with JWT
- **Relayed Room** — For lobby and matchmaking scenarios

Room hierarchy:

```

LobbyRoom (channel selection, world list)
├── WorldRoom (persistent game world per channel)
│   ├── SimulationHandler (NPCs, spawns, weather)
│   ├── PlayerHandler (movement, skills, combat)
│   ├── InventoryHandler (items, equipment, trading)
│   ├── ChatHandler (global, party, whisper)
│   └── SpawnHandler (monster respawns, loot)

```

3.4 PlayCanvas

Role: 3D game engine for the client.

Engine Version: 1.65+

Key configurations:

- **Renderer:** WebGPU with WebGL 2 fallback
- **Physics:** Ammo.js (Bullet physics via WASM)
- **Audio:** Web Audio API with spatial audio
- **Asset Pipeline:** playcanvas-sync CLI for asset uploading

Client architecture:

```

Engine Layer
├── Renderer (PBR, shadows, post-processing)
├── Physics World (collision detection, raycasting)
├── Audio Manager (SFX, ambient, music)
└── Input Manager (keyboard, mouse, gamepad)

Game Layer
├── NetworkManager (Colyseus client integration)
├── EntityManager (player, NPC, item entities)
├── UIManager (PlayCanvas Element-based UI)
├── CameraController (third-person, chase, free)
├── AnimationController (blend trees, state machines)
├── VFXManager (particles, trails, impacts)
└── AudioController (footsteps, combat, ambient)

Data Layer
├── AssetCache (textures, models, sounds)
├── ConfigLoader (game balance data from JSON)
├── LocalStorageManager (settings, cache)
└── I18nManager (localization strings)

```

3.5 Free Asset Sources

Type	Source	License
Character Models	Mixamo, Sketchfab (CC0)	Free / CC0
Environment	PolyHaven, AmbientCG	CC0
UI Icons	game-icons.net	CC BY / Public Domain
Sound Effects	Freesound.org (CC0)	CC0

Type	Source	License
Music	Incompetech, OpenGameArt	CC BY
Animations	Mixamo (automatic rigging)	Free
Particle Textures	Kenney Game Assets	CC0

4. Game Systems Design

4.1 Character System

4.1.1 Class Progression

Beginner (Level 1–15)

- Mercenary (Melee DPS) → Knight / Blade
- Acrobat (Ranged DPS) → Jester / Ranger
- Magician (Magic DPS) → Psykeeper / Elementor
- Assist (Support) → Ringmaster / Billposter

4.1.2 Stats

Stat	Description	Scale
STR	Physical attack power	+2 ATK per point
STA	Max HP, defense	+50 HP per point
DEX	Accuracy, evasion, crit	+0.5% crit per point
INT	Magic attack, MP	+2 MATK per point
SPR	Max MP, healing power	+30 MP per point

4.1.3 Leveling Formula

XP Required = $\text{floor}(\text{level}^3 * 10 + 50)$

Level	XP Required	Notable Unlock
1	60	Basic attack
5	1,500	First skill point
10	10,500	Mount system
15	34,000	Class change quest
20	80,500	Party system
30	270,500	Guild system
50	1,250,500	Elite skills
60	2,160,500	Endgame content

4.2 Combat System

4.2.1 Architecture

```

PlayerAction → ClientInput (Keyboard/Mouse)
  → NetworkManager.send("action", data)
  → Colyseus Room.handle("action")
  → CombatHandler.validate(action)
  → SkillSystem.calculate(target, skill, attacker)
  → DamageSystem.apply(attacker, target, damage)
  → State Sync (broadcast to all clients)
  → Client receives state patch
  → PlayCanvas Entity updates (animation, VFX, HP bar)

```

4.2.2 Damage Formula

Physical Damage = $(\text{ATK} * \text{SkillMultiplier} - \text{DEF} * 0.5) * \text{RandomRange}(0.9, 1.1) * \text{LevelBonus}$
 Magic Damage = $(\text{MATK} * \text{SkillMultiplier} - \text{MDEF} * 0.5) * \text{RandomRange}(0.9, 1.1) * \text{LevelBonus}$
 Crit Damage = Damage * 2.0 (triggered when DEX check passes)

4.2.3 Skills

Skill	Type	Effect	Cooldown
Bash (Merc)	Single target	150% ATK	3s
Axe Dance (Merc)	AoE	120% ATK, 3m radius	8s
Arrow Rain (Acro)	AoE	100% ATK, 5m radius	10s
Mental Strike (Mage)	Single	180% MATK	4s
Heal (Assist)	Single	300% SPR HP	6s
Haste (Assist)	Buff	+30% move speed, 30s	60s

4.3 Loot & Inventory System

4.3.1 Inventory Architecture

Inventory (9×6 = 54 slots, type: Record<number, ItemStack | null>)

- Equipment Slots (Weapon, Helmet, Armor, Gloves, Boots, Cloak, Ring x2, Necklace)
- Consumables (potions, scrolls, food)
- Materials (ore, herbs, gems)
- Quest Items (non-tradeable)
- Cash Items (premium)

4.3.2 Item Schema

```
interface Item {
  id: string;           // Unique instance ID
  templateId: number;   // Reference to ItemTemplate
  slot: number;         // Inventory slot index
  quantity: number;     // Stack size (1 for equipment)
  enchant?: number;     // Enchant level (0-15)
  durability: number;   // Current durability
  stats?: {             // Random bonus stats
    str?: number;
    sta?: number;
    dex?: number;
    int?: number;
  };
};
```

4.3.3 Loot Table System

```
interface LootTable {
  tableId: number;
  drops: Array<{
    itemTemplateId: number;
    probability: number; // 0.0 - 1.0
    minLevel: number;
    maxLevel: number;
    quantityMin: number;
    quantityMax: number;
  }>;
}
```

Monster	Common (60%)	Uncommon (30%)	Rare (9%)	Legendary (1%)
Pukepuke	Feather (1-3)	Leather (1)	Gold (100 Penya)	Pukepuke Earring

Monster	Common (60%)	Uncommon (30%)	Rare (9%)	Legendary (1%)
Mong	Fur (1-2)	Meat (1)	Mong Tooth	Mong's Fang Dagger
Lawolf	Wolf Hide (1)	Sharp Fang (1)	Lawolf Mane	Lawolf Leather Armor
Giant Guard	Iron Ore (1-2)	Steel Plate (1)	Guard Shield	Guard Commander Badge

4.4 Chat System

4.4.1 Channels

Channel	Range	Color	Cooldown
General	Current map	White	—
Party	Party members	Blue	—
Guild	Guild members	Green	—
Whisper	Specific player	Pink	—
World	All players on server	Yellow	3s
Shout	All players	Red	10s

4.4.2 Message Flow

```
Client ChatInput → NetworkManager.send("chat:message", { channel, text })
  → ChatHandler.validate (length, spam filter, profanity filter)
  → ChatHandler.route(channel)
  → GameServer.broadcast (or direct for whisper)
  → Redis Pub/Sub for cross-channel relay
  → Client receives and renders via ChatPanel UI
```

4.5 Party System

4.5.1 Party Structure

```
interface Party {
  id: string;
  leaderId: string;
  members: PartyMember[];
  maxSize: 6;
  lootMode: 'free' | 'order' | 'random';
  shareLevelRange: 20; // Must be within 20 levels to share XP
}

interface PartyMember {
  sessionId: string;
  characterId: string;
  name: string;
  level: number;
  class: ClassType;
  health: number;
  maxHealth: number;
  mana: number;
  maxMana: number;
  mapId: string;
  online: boolean;
}
```

4.5.2 XP Bonus

Party XP Bonus = (MemberCount - 1) * 0.1 (max 50% bonus)
 Individual XP Share = MonsterXP * (Level^2 / SumOfAllMemberLevels^2) * PartyBonus

4.6 Flight / Mount System

4.6.1 Mount Types

Mount	Level	Req Speed	Acquired
Apprentice Broom	10	150%	Quest reward
Starter Board	10	140%	NPC shop (10,000 Penny)
Speed Broom	30	200%	Crafting
Premium Board	30	210%	Cash shop
Rare Broom	50	260%	Boss drop

4.6.2 Flight Mechanics

- **Fuel system:** Flight consumes Stamina (regenerates while grounded)
- **Altitude:** 3 tiers — Ground, Low Flight (5m), High Flight (20m)
- **Combat:** Aerial combat supported at Low Flight
- **Collision:** Slide along terrain/structures, cannot clip through

4.7 Lobby & Matchmaking

4.7.1 Lobby Flow

Client Login
 → Character Select Screen
 → Channel Select (Flarine / Saint Morning / Darken)
 → Enter World (Colyseus Room)
 → Spawn at Last Save Point

4.7.2 Channel System

- Each server has 3-5 channels
- Channels share the same world data but have separate Colyseus Room instances
- Players can switch channels via NPC with 30-second cooldown
- Channel capacity: 200 players each

4.8 Dashboard & Leaderboard

4.8.1 Dashboard Metrics

Section	Metrics
Profile	Level, class, playtime, guild, title
Combat	Total kills, PvP kills, PvE kills, death count
Economy	Total Penny earned, spent, items crafted
Social	Party count, guild contribution, friends added
Achievements	Badge count, completion percentage

4.8.2 Leaderboard Tiers

Tier	Criteria	Reward
Level	Character level	Title: "Legendary"
PVP	PvP kill count	Title: "Gladiator"
Wealth	Total Penny	Title: "Tycoon"
Guild	Guild level	Guild bonus XP

Tier	Criteria	Reward
Fame	Achievement points	Title: "Hero"

4.9 NPC & Quest Systems

4.9.1 NPC Types

Type	Behavior	Example
Shop	Sells items/buys items	Weapon Merchant
Quest	Gives/completes quests	Guide NPC
Storage	Personal bank access	Banker
Skill	Teaches skills	Skill Master
Class	Handles class changes	Class Master
Transport	Teleport between maps	Flarine Transport

4.9.2 Quest Types

Type	Objective	Rewards
Kill	Defeat N monsters	XP, Penya
Collect	Gather N items	XP, Items
Delivery	Take item to NPC	XP, Reputation
Escort	Protect NPC	XP, Title
Daily	Repeatable daily	XP, Tokens
Chain	Series of quests	Equipment
Class	Advancement quest	New class

5. Network Architecture (Colyseus)

5.1 Room Schema Definitions

```
// Shared state schema
class PlayerState extends Schema {
  @type('string') id: string;
  @type('string') name: string;
  @type('int8') level: number;
  @type('float32') x: number;
  @type('float32') y: number;
  @type('float32') z: number;
  @type('float32') rotation: number;
  @type('int16') hp: number;
  @type('int16') maxHp: number;
  @type('int16') mp: number;
  @type('int16') maxMp: number;
  @type('string') class: string;
  @type('string') mount: string;
  @type('boolean') flying: boolean;
  @type('int8') altitude: number;
}

class MonsterState extends Schema {
  @type('string') id: string;
  @type('int16') templateId: number;
  @type('float32') x: number;
  @type('float32') y: number;
  @type('float32') z: number;
  @type('float32') rotation: number;
  @type('int16') hp: number;
  @type('int16') maxHp: number;
}
```

```

    @type('string') aiState: string;
    @type('string') targetId: string;
}

class WorldState extends Schema {
    @type({ map: PlayerState }) players = new MapSchema<PlayerState>();
    @type({ map: MonsterState }) monsters = new MapSchema<MonsterState>();
    @type('float32') time: number; // Day/night cycle
    @type('string') weather: string;
    @type('int16') playerCount: number;
}

```

5.2 Message Protocol

Message	Direction	Payload
player:move	C→S	{ x, y, z, rotation }
player:jump	C→S	{ }
player:action	C→S	{ type: 'attack'
player:chat	C→S	{ channel, text }
player:emote	C→S	{ emoteId }
inventory:move	C→S	{ fromSlot, toSlot }
inventory:use	C→S	{ slot, targetId? }
inventory:equip	C→S	{ slot }
inventory:drop	C→S	{ slot, quantity }
trade:request	C→S	{ targetId }
trade:accept	C→S	{ tradeId }
trade:addItem	C→S	{ tradeId, slot, quantity }
trade:confirm	C→S	{ tradeId }
party:invite	C→S	{ targetId }
party:accept	C→S	{ partyId }
party:leave	C→S	{ }
party:kick	C→S	{ memberId }
guild:create	C→S	{ name }
guild:invite	C→S	{ targetId }
guild:accept	C→S	{ guildId }

5.3 Server Tick Loop

20 Hz Server Tick:

1. ProcessInputQueue (player actions)
2. UpdateMovement (position validation, speed hack detection)
3. UpdateAI (monster behavior trees)
4. UpdateCombat (skill cooldowns, damage processing)
5. UpdateSpawns (monster respawns, loot cleanup)
6. UpdateEnvironment (day/night, weather)
7. UpdateBuffs (tick buffs/debuffs)
8. PersistDirtyData (write to DB if needed)
9. EndStatePatch – Colyseus broadcasts delta to clients

5.4 Lag Compensation

- **Client-side prediction** for movement
- **Server reconciliation** with authoritative position correction
- **Input buffering** — store last 100ms of inputs, reapply on state correction
- **Interpolation** — render other entities at interpolated positions between state patches

6. Client Architecture (PlayCanvas + TypeScript)

6.1 Entity Component System

```
// Base entity pattern
class GameEntity {
  entity: pc.Entity;
  private components: Map<string, IComponent>;

  addComponent<T extends IComponent>(type: string, component: T): void;
  getComponent<T extends IComponent>(type: string): T | undefined;
  removeComponent(type: string): void;
  update(dt: number): void;
}

// Component examples
interface IComponent {
  entity: GameEntity;
  initialize(): void;
  update(dt: number): void;
  destroy(): void;
}

class HealthComponent implements IComponent {
  current: number;
  max: number;
  regenRate: number;

  takeDamage(amount: number, source: GameEntity): void;
  heal(amount: number): void;
  tick(dt: number): void; // Regen
}

class MovementComponent implements IComponent {
  speed: number;
  velocity: pc.Vec3;
  isFlying: boolean;
  altitude: number;

  moveTo(target: pc.Vec3, dt: number): void;
  stop(): void;
  setFlight(enabled: boolean, altitude: number): void;
}

class InventoryComponent implements IComponent {
  slots: (ItemStack | null)[];
  equipment: EquipmentMap;
  gold: number;

  addItem(item: Item): boolean;
  removeItem(slot: number): void;
  equip(slot: number): boolean;
  unequip(slot: EquipmentSlot): void;
  hasSpace(): boolean;
}
```

6.2 Network Manager

```
class NetworkManager {
  private client: Colyseus.Client;
  private room: Room<WorldState>;
  private pendingActions: QueuedAction[];
  private serverTime: number;

  async connect(authToken: string): Promise<void>;
  async joinWorld(worldId: string): Promise<void>;
  send(action: string, data: any): void;
  onStateChange(callback: (state: WorldState) => void): void;
  onMessage(type: string, callback: (data: any) => void): void;
  synchronize(): void; // Apply pending state patches
}
```

```

    getLatency(): number;
    getServerTime(): number;
}

```

6.3 UI System

```

// UI Panel base class
abstract class UIPanel {
    protected entity: pc.Entity;
    protected screen: pc.ScreenComponent;
    protected visible: boolean;

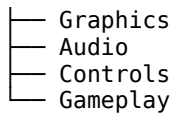
    abstract onOpen(): void;
    abstract onClose(): void;
    abstract refresh(): void; // Update displayed data
    show(): void;
    hide(): void;
    toggle(): void;
    isVisible(): boolean;
}

```

```

// Panel hierarchy
├── HUD
│   ├── HealthBar
│   ├── ManaBar
│   ├── ExpBar
│   ├── SkillBar (1-0 hotkeys)
│   ├── Minimap
│   ├── BuffBar
│   └── TargetInfo
├── InventoryPanel
│   ├── ItemGrid (54 slots)
│   ├── EquipmentPanel
│   ├── GoldDisplay
│   └── ItemTooltip
├── CharacterPanel
│   ├── StatsDisplay
│   ├── EquipmentViewer
│   └── TitleDisplay
├── ChatPanel
│   ├── ChatLog
│   ├── ChannelTabs
│   └── InputField
├── PartyPanel
│   ├── MemberList
│   ├── MemberHP/MP bars
│   └── LeaveButton
├── QuestPanel
│   ├── QuestList
│   ├── QuestDetails
│   └── TrackedQuest
├── ShopPanel
│   ├── ItemList
│   ├── Buy/Sell tabs
│   └── PriceDisplay
├── SkillPanel
│   ├── SkillTree
│   ├── SkillSlots
│   └── SkillDescription
├── TradePanel
│   ├── MyItems
│   ├── TheirItems
│   └── ConfirmButton
├── LobbyPanel
│   ├── CharacterSelect
│   ├── WorldSelect
│   └── CreateCharacter
├── DashboardPanel
│   ├── ProfileStats
│   ├── Achievements
│   └── LeaderboardTable
└── SettingsPanel

```



6.4 Camera System

```

class CameraController {
    private camera: pc.Entity;
    private target: pc.Entity;
    private mode: 'thirdperson' | 'firstperson' | 'free';

    // Third-person
    private distance: number = 5;
    private height: number = 2;
    private pitch: number = -15;
    private yaw: number = 0;
    private zoomSpeed: number = 2;
    private sensitivity: number = 0.15;
    private collisionRadius: number = 0.3;

    update(dt: number): void;
    zoom(delta: number): void;
    rotate(pitchDelta: number, yawDelta: number): void;
    setMode(mode: CameraMode): void;
    lookAt(target: pc.Vec3): void;
    handleCollision(): void; // Camera collision with world
}

```

7. Security & Anti-Cheat

7.1 Server Authority

All game-critical logic is **server-authoritative**:

- **Movement**: Client sends inputs, server validates against physics
- **Combat**: Server calculates damage, hit validation, cooldowns
- **Loot**: Server generates loot tables, validates pickup
- **Economy**: All currency transactions validated server-side
- **Leveling**: XP awarded and levels calculated server-side

7.2 Anti-Cheat Measures

Threat	Mitigation
Speed hacking	Server validates max movement delta per tick (20 Hz = max 50ms position change)
Teleport hacks	Reject positions > max move speed from last known
Memory editing	Server-side stats (no trust in client values)
Packet manipulation	Sequence numbers, checksums, replay protection
Auto-target bots	CAPTCHA for repetitive actions, rate limiting
Gold duping	Transaction log, server-authoritative inventory
Wallhack	Server-side line-of-sight checks for combat
Speed/Fly hacks	Server validates flight state transitions

7.3 Encryption & Authentication

Login Flow:

```

Client → Password hashed with bcrypt (client-side salt)
Server → Verify bcrypt hash, issue JWT

```

Client → Connect to Game Server with JWT in headers
 Game Server → Verify JWT signature with shared secret
 Game Server → Load character data from DB

In-Game Security:

- Message sequence numbers (detect replay attacks)
- Messages signed with session-derived key
- Rate limiting per message type
- IP reputation tracking
- Concurrent session detection

7.4 Rate Limiting

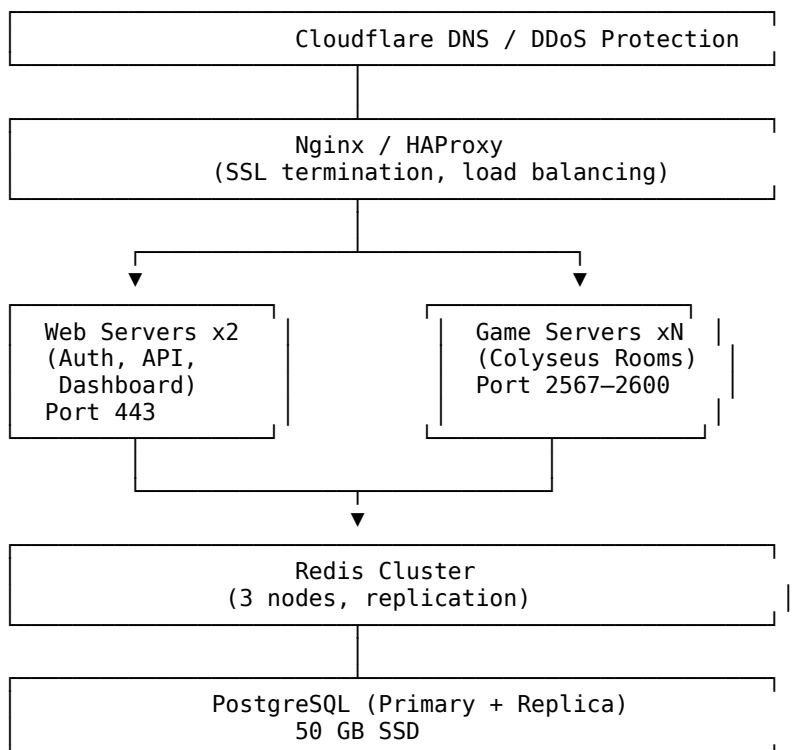
Action	Limit	Ban Threshold
Login attempts	5/minute	20/minute
Chat messages	5/second	30/second
Attack actions	10/second	30/second
Item pickups	5/second	-
Trade requests	3/minute	-
Party invites	5/minute	-

7.5 Logging & Monitoring

- **Audit log:** All server-authoritative actions logged
- **Anomaly detection:** Flag unusual patterns (leveling too fast, impossible damage)
- **Auto-ban system:** Temp ban on suspicious activity threshold
- **Admin tools:** In-game GM commands for investigation

8. Deployment Architecture

8.1 Infrastructure Diagram



8.2 Server Specs

Tier	CPU	RAM	Storage	Bandwidth	Count
Web Server	4 vCPU	8 GB	50 GB SSD	1 Gbps	2
Game Server	8 vCPU	16 GB	50 GB SSD	1 Gbps	3
Redis	4 vCPU	8 GB	20 GB SSD	1 Gbps	3
PostgreSQL	8 vCPU	16 GB	100 GB SSD	1 Gbps	2

8.3 Docker Compose (Local Development)

```
version: '3.8'
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: flyff_mvp
      POSTGRES_USER: flyff
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  auth-server:
    build: ./server/auth
    environment:
      - REDIS_URL=redis://redis:6379
      - DATABASE_URL=postgresql://flyff:${DB_PASSWORD}@postgres:5432/flyff_mvp
      - JWT_SECRET=${JWT_SECRET}
    depends_on: [postgres, redis]

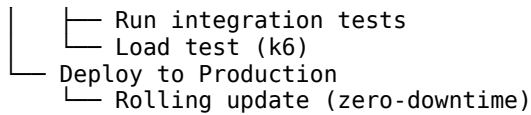
  game-server:
    build: ./server/game
    environment:
      - REDIS_URL=redis://redis:6379
      - DATABASE_URL=postgresql://flyff:${DB_PASSWORD}@postgres:5432/flyff_mvp
      - JWT_SECRET=${JWT_SECRET}
    ports:
      - "2567:2567"
    depends_on: [postgres, redis]

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./deploy/nginx.conf:/etc/nginx/nginx.conf
    depends_on: [auth-server, game-server]

volumes:
  pgdata:
```

8.4 CI/CD Pipeline

```
Git Push → GitHub Actions
├── Lint (ESLint + Prettier)
├── Type Check (tsc --noEmit)
├── Unit Tests (Vitest)
├── Build
│   ├── Server → Docker image
│   └── Client → PlayCanvas publish
└── Deploy to Staging
```



8.5 Monitoring & Observability

Tool	Purpose
Prometheus	Metrics collection (CPU, memory, players, room count)
Grafana	Dashboards, alerts
Winston	Structured logging (JSON format)
Sentry	Error tracking
Colyseus Monitor	Web dashboard for room inspection

8.6 Scaling Strategy

Horizontal Scaling:

- Add more Game Server instances behind load balancer
- Increase Redis cluster size
- Add PostgreSQL read replicas for dashboard queries

Vertical Scaling:

- Increase server CPU/memory for higher player caps per room
- SSDs for database performance

Auto-scaling triggers:

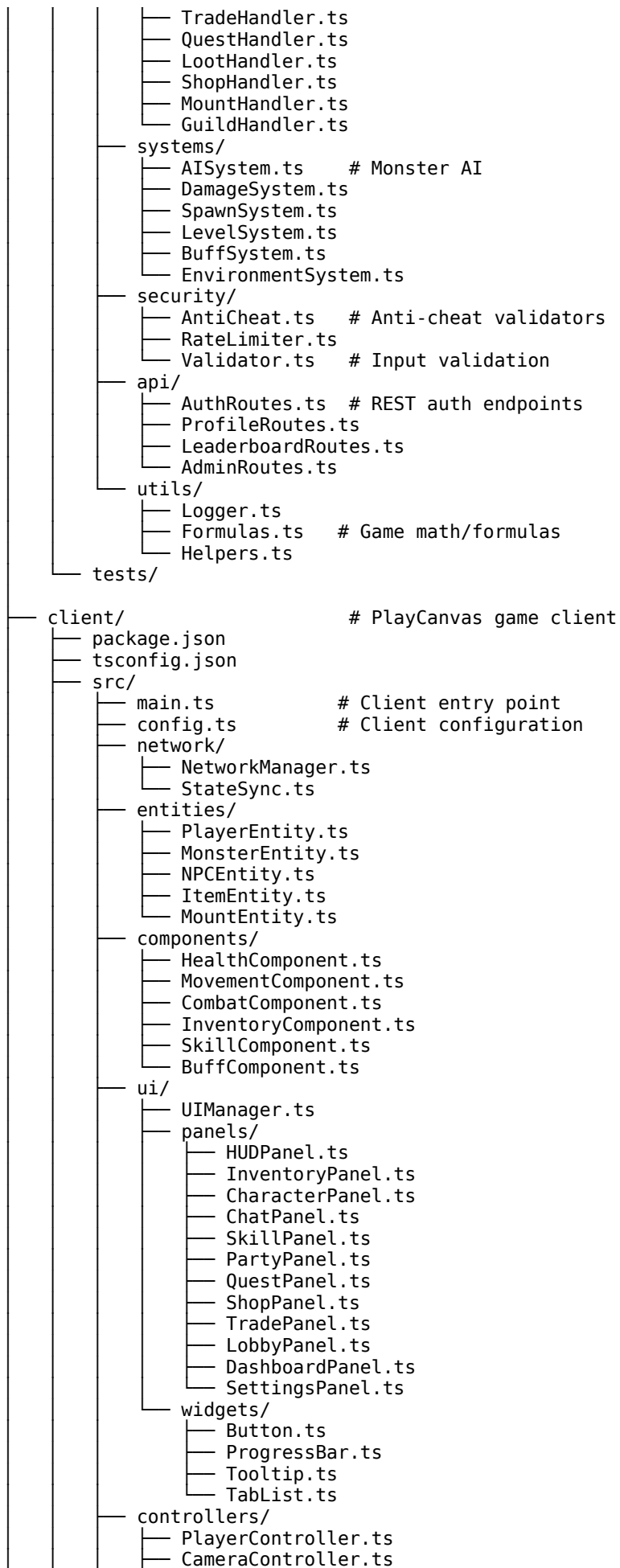
- CPU > 70% for 5 minutes → spawn new game server
- Player count > 800 per server → spawn new channel
- Redis memory > 80% → trigger cluster resize

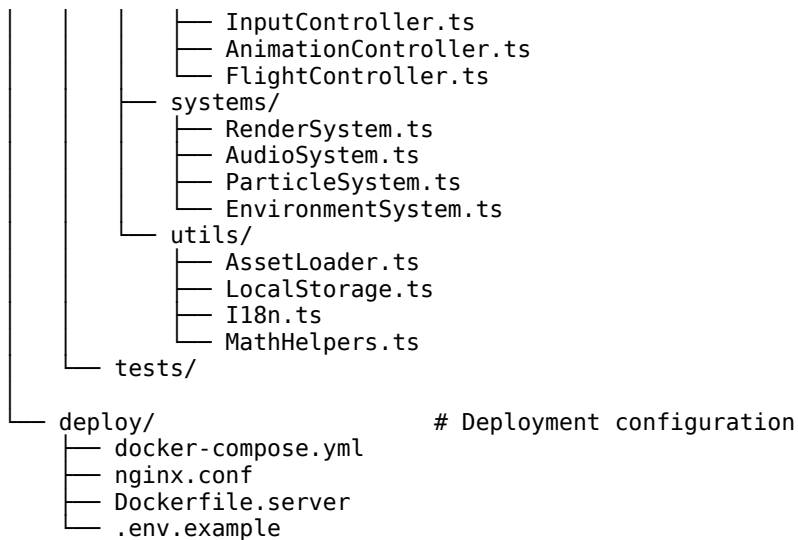
9. Code File Reference

All source code files for the FlyFF MVP are organized under `/Plan/Code/` with the following structure:

```

/Plan/Code/
├── shared/                                # Shared TypeScript types
│   ├── types.ts                          # Core type definitions
│   ├── constants.ts                      # Game constants
│   └── interfaces.ts                     # Shared interfaces
├── server/                               # Node.js + Colyseus server
│   ├── package.json
│   ├── tsconfig.json
│   └── src/
│       ├── index.ts                      # Server entry point
│       ├── config.ts                     # Server configuration
│       └── database/
│           ├── connection.ts             # Database connection
│           ├── entities/                 # TypeORM entities
│           └── migrations/               # Database migrations
│       ├── auth/
│           ├── AuthService.ts             # Authentication logic
│           └── AuthRoom.ts               # Colyseus auth room
│       ├── rooms/
│           ├── LobbyRoom.ts              # Channel/world selection
│           ├── WorldRoom.ts              # Main game world room
│           └── ChatRoom.ts               # Chat-only room
│       └── handlers/
│           ├── MovementHandler.ts
│           ├── CombatHandler.ts
│           ├── InventoryHandler.ts
│           ├── SkillHandler.ts
│           ├── PartyHandler.ts
│           └── ChatHandler.ts
  
```



9.1 File Descriptions

shared/types.ts

Core TypeScript type definitions shared between client and server. Includes enums for classes, item types, chat channels, etc. Used to ensure type consistency across the entire codebase.

shared/constants.ts

Game balance constants — XP tables, stat growth curves, skill cooldowns, movement speeds, inventory limits, channel capacities. Centralized tuning file.

server/src/index.ts

Server entry point. Configures Express + Colyseus, registers room handlers, connects to database/Redis, starts HTTP server. Also mounts admin dashboard.

server/src/rooms/WorldRoom.ts

The core game room. Contains the main game loop (20 Hz tick), manages player lifecycle, delegates to handlers for specific gameplay domains. Maintains the authoritative `WorldState` schema.

server/src/handlers/CombatHandler.ts

Handles attack actions, validates targets, calculates damage using `DamageSystem`, applies damage to targets, broadcasts results. Checks line-of-sight and range server-side.

server/src/handlers/MovementHandler.ts

Validates and processes player movement inputs. Checks for speed hacks, teleportation, out-of-bounds. Updates player position in `WorldState`.

server/src/systems/DamageSystem.ts

Pure function math — calculates final damage considering ATK/MATK, DEF/MDEF, skill multipliers, level bonuses, critical hits, and damage variance.

server/src/systems/AISystem.ts

Behavior tree-based monster AI. States: IDLE, PATROL, CHASE, ATTACK, FLEE, RETURN. Uses configurable detection ranges from monster templates.

client/src/network/NetworkManager.ts

Manages the WebSocket connection to the Colyseus server. Handles reconnection, message serialization, latency measurement, and state patch application.

client/src/entities/PlayerEntity.ts

Wraps a PlayCanvas entity with all player-specific behaviors: network state sync, animation blending, mount display, nameplate, HP bar overhead.

client/src/ui/panels/HUDPanel.ts

Main in-game HUD: health/mana/XP bars, skill bar, minimap, buffs, target info. Created once on world entry, refreshed every frame from local entity state.

client/src/controllers/PlayerController.ts

Handles local player input (WASD movement, mouse look, action keys). Uses client-side prediction for immediate feedback, sends inputs to server for reconciliation.

client/src/controllers/CameraController.ts

Third-person camera with collision detection, zoom, mouse orbit. Supports flight mode with dynamic distance and pitch based on altitude.

deploy/docker-compose.yml

Single-command local development environment. Spins up PostgreSQL, Redis, auth server, game server, and Nginx reverse proxy.

Appendix A: Database Schema (PostgreSQL)

Accounts Table

```
CREATE TABLE accounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(32) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  salt VARCHAR(64) NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),
  last_login TIMESTAMP,
  is_banned BOOLEAN DEFAULT FALSE,
  is_admin BOOLEAN DEFAULT FALSE
);
```

Characters Table

```
CREATE TABLE characters (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  account_id UUID REFERENCES accounts(id),
  name VARCHAR(16) UNIQUE NOT NULL,
  class VARCHAR(32) NOT NULL DEFAULT 'Beginner',
  level INTEGER NOT NULL DEFAULT 1,
  experience BIGINT NOT NULL DEFAULT 0,
  str INTEGER NOT NULL DEFAULT 5,
  sta INTEGER NOT NULL DEFAULT 5,
```

```

dex INTEGER NOT NULL DEFAULT 5,
int INTEGER NOT NULL DEFAULT 5,
spr INTEGER NOT NULL DEFAULT 5,
stat_points INTEGER NOT NULL DEFAULT 0,
hp INTEGER NOT NULL DEFAULT 100,
max_hp INTEGER NOT NULL DEFAULT 100,
mp INTEGER NOT NULL DEFAULT 50,
max_mp INTEGER NOT NULL DEFAULT 50,
gold BIGINT NOT NULL DEFAULT 0,
map_id VARCHAR(64) NOT NULL DEFAULT 'Flarine',
position_x REAL NOT NULL DEFAULT 0,
position_y REAL NOT NULL DEFAULT 0,
position_z REAL NOT NULL DEFAULT 0,
rotation REAL NOT NULL DEFAULT 0,
created_at TIMESTAMP DEFAULT NOW(),
last_saved TIMESTAMP DEFAULT NOW(),
playtime_seconds BIGINT DEFAULT 0,
deletion_mark BOOLEAN DEFAULT FALSE
);

```

Items Table

```

CREATE TABLE items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  character_id UUID REFERENCES characters(id),
  template_id INTEGER NOT NULL,
  slot INTEGER NOT NULL,
  quantity INTEGER NOT NULL DEFAULT 1,
  enchant_level INTEGER DEFAULT 0,
  durability INTEGER DEFAULT 100,
  bonus_str INTEGER DEFAULT 0,
  bonus_sta INTEGER DEFAULT 0,
  bonus_dex INTEGER DEFAULT 0,
  bonus_int INTEGER DEFAULT 0,
  is_equipped BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW()
);

```

Guilds Table

```

CREATE TABLE guilds (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(24) UNIQUE NOT NULL,
  leader_id UUID REFERENCES characters(id),
  level INTEGER DEFAULT 1,
  experience BIGINT DEFAULT 0,
  member_count INTEGER DEFAULT 1,
  max_members INTEGER DEFAULT 20,
  created_at TIMESTAMP DEFAULT NOW(),
  emblem_data TEXT
);

```

Leaderboard Snapshot Table

```

CREATE TABLE leaderboard_snapshots (
  id BIGSERIAL PRIMARY KEY,
  snapshot_type VARCHAR(16) NOT NULL, -- 'level', 'pvp', 'wealth'
  character_id UUID REFERENCES characters(id),
  value BIGINT NOT NULL,
  updated_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_leaderboard_type_value ON leaderboard_snapshots(snapshot_type, value DESC);

```

Appendix B: Performance Targets

Area	Metric	Target
Network	Bandwidth per player	< 10 KB/s
Network	State patch size	< 2 KB
Server	Msg processing per tick	< 10 ms
Server	Tick duration	< 50 ms (20 Hz)
Client	Draw calls	< 500
Client	Memory usage	< 512 MB
Client	Loading time (initial)	< 10 seconds
DB	Query response	< 20 ms
DB	Write batch interval	5 seconds

Appendix C: Free Asset Integration Plan

Asset Category	Source URL	Integration Method
Character base mesh	Mixamo (Fuse) → FBX → PlayCanvas pipeline	Manual conversion via PlayCanvas Editor
Animations	Mixamo auto-rigging → FBX	Upload via PlayCanvas Asset Pipeline
Environment textures	PolyHaven (2K PBR)	Download → Convert → PlayCanvas material
Skybox	OpenGameArt (HDRI)	HDR → CubeMap → PlayCanvas Skybox
UI icons	game-icons.net (SVG)	SVG → PNG sprite sheet → PlayCanvas Element
Sound effects	Freesound.org CC0	MP3 → OGG → PlayCanvas Audio Asset
Music tracks	Incompetech	MP3 → OGG → Streaming audio
Particle textures	Kenney Particle Pack	PNG → PlayCanvas ParticleAsset
Weapon models	Sketchfab CC0	GLB → PlayCanvas Model Asset

End of Document